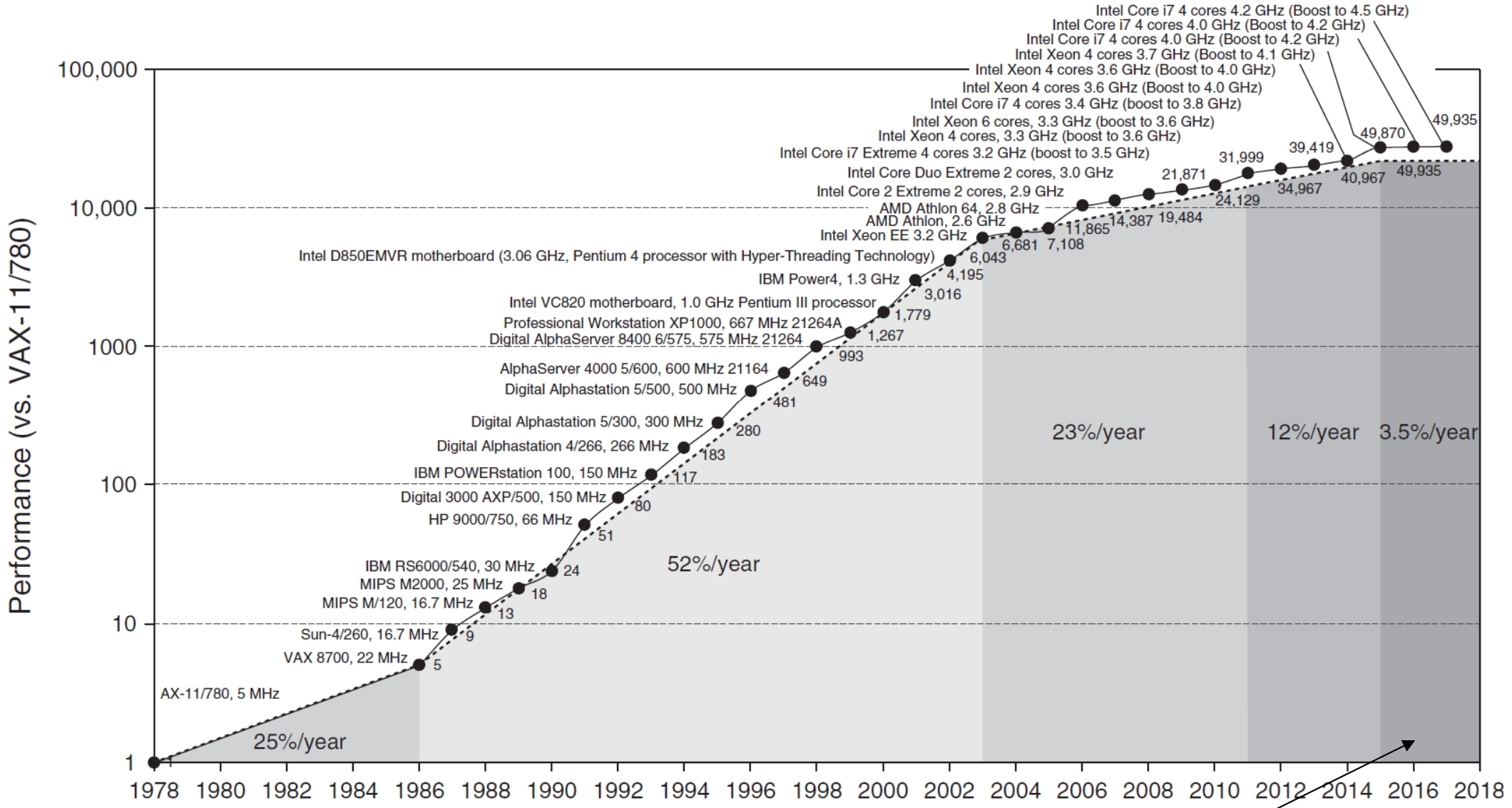
Evoluzione dei processor con pipelined

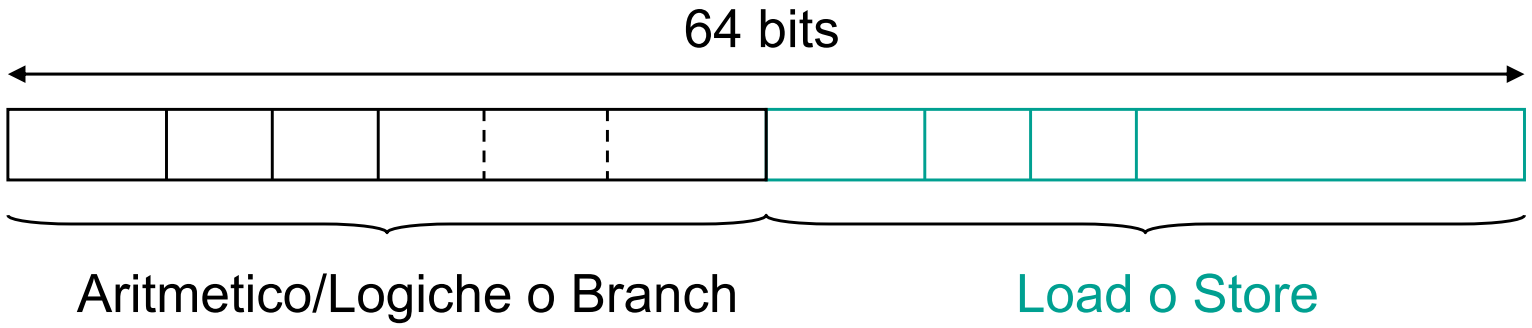
	Anno	Clock Rate	# Pipe Stages	Issue Width	Cores/ Chip	Power
Intel 486	1989	25 MHz	5	1	1	5 W
Intel Pentium	1993	66 MHz	5	2	1	10 W
Intel Pentium Pro	1997	200 MHz	10	3	1	29 W
Intel Pentium 4 Willamette	2001	2000 MHz	22	3	1	75 W
Intel Pentium 4 Prescott	2004	3600 MHz	31	3	1	103 W
Intel Core	2006	2930 MHz	14	4	2	75 W
Sun USPARC III	2003	1950 MHz	14	4	1	90 W
Sun T1 (Niagara)	2005	1200 MHz	6	1	8	70 W



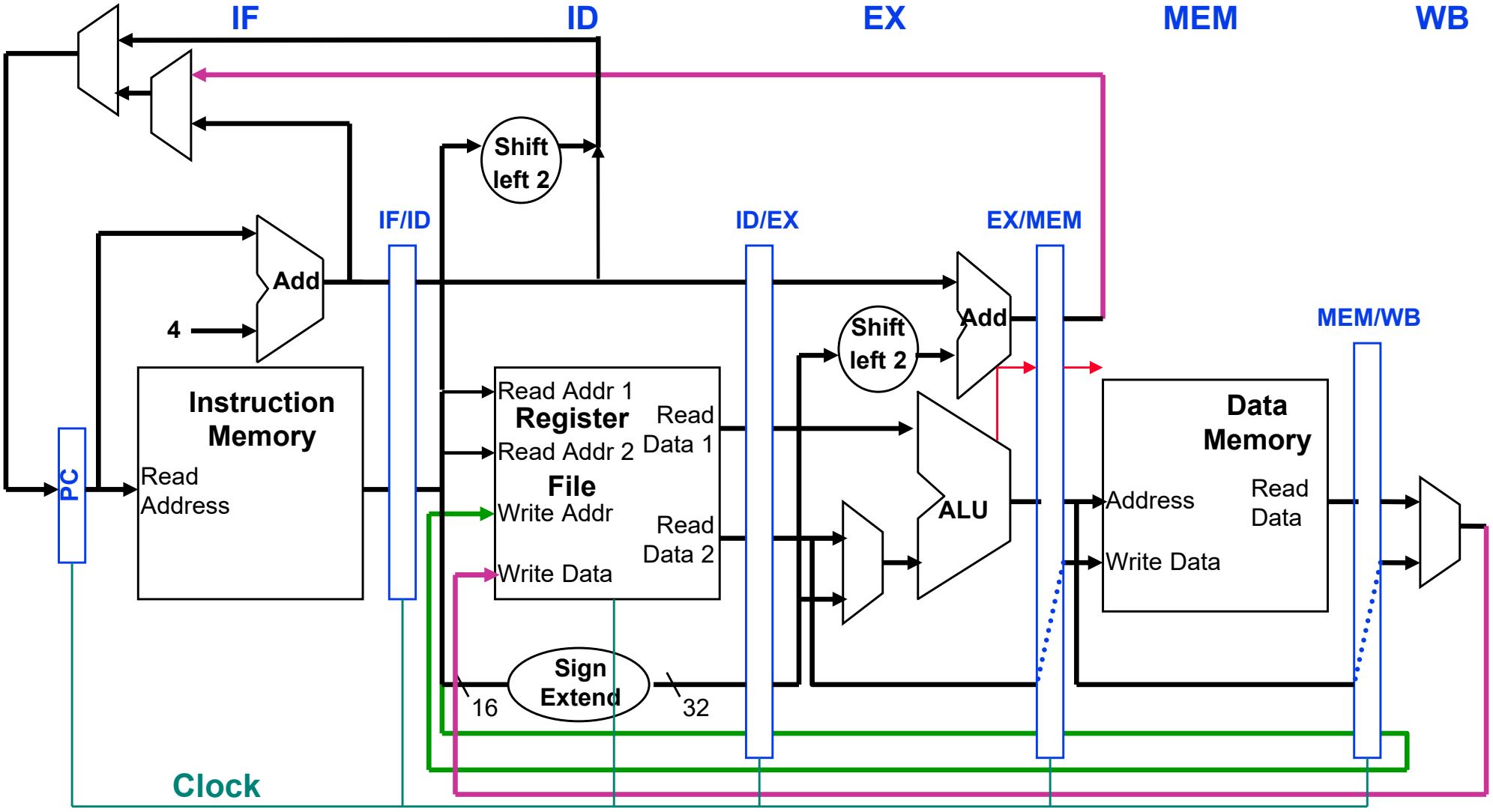
LIMITATI da : consumi/dissipazione, instruction-level parallelism, memory latency

Very Long Instruction Word / Static Multiple Issue

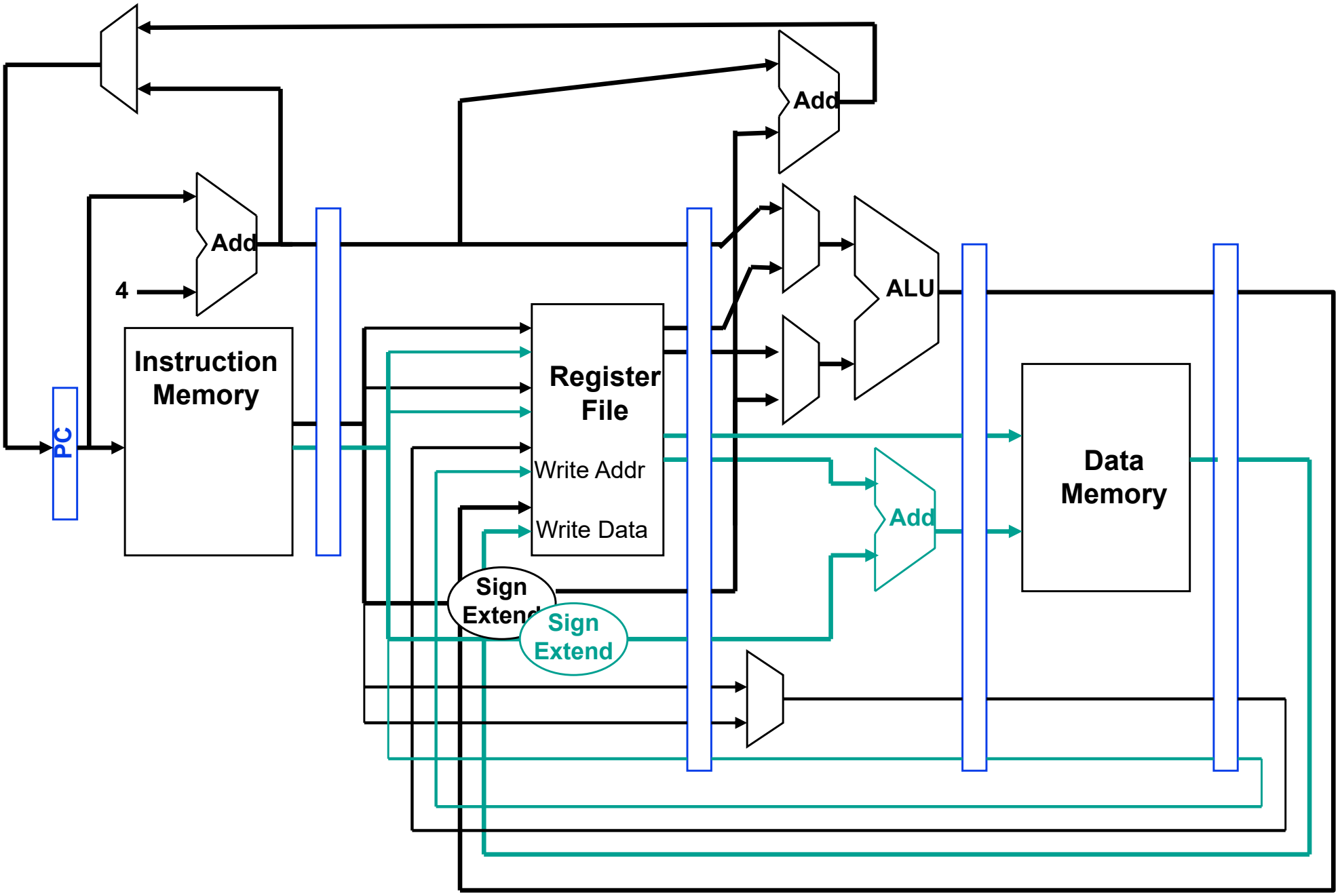
- Esempio: 2-issue



Pipeline Datapath



VLIW (2-issue) Datapath



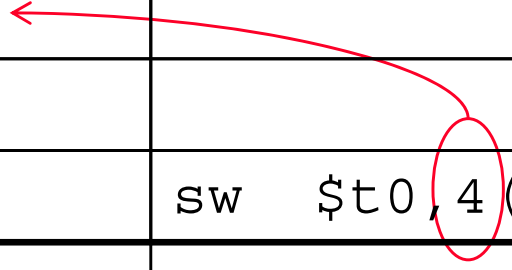
Esempio: loop

```
lp:   lw    $t0,0($s1)    # $t0 = elemento array
      add   $t0,$t0,$s2   # $t0 = $t0 + $s2
      sw    $t0,0($s1)    # store result
      addi  $s1,$s1,-4    # decrementa puntatore
      bne   $s1,$0,lp     # branch if $s1 != 0
```

Schedulazione istruzioni in coppie

```
lp:   lw   $t0,0($s1)   # $t0 = elemento array
      add  $t0,$t0,$s2  # $t0 = $t0 + $s2
      sw   $t0,0($s1)   # store result
      addi $s1,$s1,-4   # decrementa puntatore
      bne  $s1,$0,lp    # branch if $s1 != 0
```

	ALU or branch	Data transfer	CC
lp:		lw $t0,0($s1)	1
	addi $s1,$s1,-4		2
	add $t0,$t0,$s2		3
	bne $s1,$0,lp	sw $t0,4($s1)	4
			5



Loop unrolling

```
#include <stdio.h>

#define N 10000

int main(int argc,char *argv[])
{
int v[N];
int i, j;
int R=atoi(argv[1]);

for(j=0; j<R; j++)
    for(i=0; i<N; i++)
        v[i]=i*j;
printf("%d %d\n", j, v[(i-1)]);
}
```

```
#include <stdio.h>


#include <stdio.h>

#define N 10000

int main(int argc,char *argv[])
{
int v[N];
int i, j;
int R=atoi(argv[1]);

for(j=0; j<R; j++)
    for(i=0; i<N; i+=10) {
        v[i]=i*j;
        v[i+1]=i*j;
        v[i+2]=i*j;
        v[i+3]=i*j;
        v[i+4]=i*j;
        v[i+5]=i*j;
        v[i+6]=i*j;
        v[i+7]=i*j;
        v[i+8]=i*j;
        v[i+9]=i*j;
        }

printf("%d %d\n", j, v[(i-1)%R]);
}
```

Esempio unrolling di quattro

```
lp:   lw   $t0,0($s1)    # $t0=array element
      lw   $t1,-4($s1)   # $t1=array element
      lw   $t2,-8($s1)   # $t2=array element
      lw   $t3,-12($s1)  # $t3=array element
      add  $t0,$t0,$s2   # add scalar in $s2
      add  $t1,$t1,$s2   # add scalar in $s2
      add  $t2,$t2,$s2   # add scalar in $s2
      add  $t3,$t3,$s2   # add scalar in $s2
      sw   $t0,0($s1)    # store result
      sw   $t1,-4($s1)   # store result
      sw   $t2,-8($s1)   # store result
      sw   $t3,-12($s1)  # store result
      addi $s1,$s1,-16   # decrement pointer
      bne  $s1,$0,lp     # branch if $s1 != 0
```

Schedulazione istruzioni in coppie

	ALU o branch	Memoria Dati	CC
lp:	addi $s1,$s1,-16	lw $t0,0($s1)	1
		lw $t1,12($s1)	2
	add $t0,$t0,$s2	lw $t2,8($s1)	3
	add $t1,$t1,$s2	lw $t3,4($s1)	4
	add $t2,$t2,$s2	sw $t0,16($s1)	5
	add $t3,$t3,$s2	sw $t1,12($s1)	6
		sw $t2,8($s1)	7
	bne $s1,$0,lp	sw $t3,4($s1)	8

- Otto cicli di clock per eseguire 14 instructions :
  - CPI = 0.57 (massimo sarebbe 0.5)
  - IPC = 1.8 (massimo sarebbe 2.0)

Dynamic Multiple Issue / Super Scalarità

- I processori decidono durante l'esecuzione (a run-time) quali istruzioni eseguire simultaneamente.

- Instruction-fetch and issue.
  Le sitruzioni vengono lette, decodificate e inserite (issued) in una coda gestita da una Front-End Unit in attesa di essere eseguite.
  Si ricorre ad "instruction lookahead" ovvero vengono lette più istruzioni oltre quella corrente.

- Instruction-execution
  Le istruzioni possono essere eseguite non appena gli operandi sono disponibili eventualmente in ordine diverso da quello originale (definito dal compilatore).

- Instruction-commit
  Evoluzione del write back: I risultati vengono scritti quando si è sicuri di poterlo fare rispettando l'ordine orginale di esecuzione delle istruzioni.

Esecuzione In-Order e Out-of-Order

- Lettura (fetch) e decodifica devono essere fatte in-order identificando tutte le dipendenze tra le istruzioni.
- La scrittura dei risultati (commit) nei registry o in memoria deve essere fatta rispettando l'ordine di lettura (in-order) e facendo attenzione a :
  - eccezioni
  - branch misprediction
- L'esecuzione può avvenire senza rispettare l'ordine fissato dal compilatore (ou-of-order)

Out-of-Order Execution

- Con l'esecuzione out-of-order occorre identificare e risolvere non solo i conflitti di tipo read before write ma anche quelli write before write :

```
lw    $t0,0($s1)
addu  $t0,$t1,$s2
. . .
sub   $t2, $t0, $s2
```

Antidependencies

- Nascono anche le anti-dipendenze – quando una istruzione successiva (che viene eseguita anticipatamente) produce un risultato che sovrascrive un valore necessario come operando di una istruzione precedente (che viene eseguita ritardata) :

```
R3 := R3 * R5
R4 := R3 + 1
R3 := R5 + 1
```

Antidependency
True data dependency
Output dependency

- Il problema è simile a quello dei normali conflitti sui dati , semplicemente rovesciato
  - Invece di avere un'istruzione che necessita di un dato non ancora disponibile perchè risultato di una precedente istruzione non ancora completata (read before write), adesso si tratta di una istruzione che sovrascrivere un dato non ancora utilizzato da una istruzione che la precede nell'ordine originale ma che non è stata ancora eseguita (write before read).

Dipendenze

- Ciascuna dipendenza sui dati
  - Dipendenze reali (read before write)
  - Anti-dipendenze (write before read)
  - Output-dipendenze (write before write)

  } Conflitti di scrittura

  si identificano in base all'uso dei registry (o di altri elementi di memoria)

- Le dipendenze reali derivano dal flusso/utilizzo dei dati all'interno di un programma/algoritmo.

- Anti e Output dipendenze nascono a causa del limitato numero di registry che costringono al loro riutilizzo. Si tratta quindi in realtà di conflitti strutturali.

Storage Conflicts and Register Renaming

- I conflitti strutturali possono essere ridotti o eliminate duplicando le risorse HW su cui si crea il conflitto.
- Quindi in questo caso si tratta di aumentare il numero di registry che però sono un elemento fissato dalla ISA.
- Soluzione: Register Renaming – I registri usati dalle istruzioni vengono cambiati dinamicamente durante l'esecuzione quando si identifica un conflitto :

```
R3 := R3 * R5          R3b := R3a * R5a
R4 := R3 + 1     ⇨     R4a := R3b + 1
R3 := R5 + 1           R3c := R5a + 1
```